tomat 🍅

Tokenizer fidelity for electron-density prediction

Ryan Williams · Open Athena · 2026-04-21

The problem

Predict the electron density `ϱ(r)` of a crystal structure with an LLM. Transformer alternative to electrAI's 3D ResUNet.

Pipeline: structure → **tokens** → Qwen3 → tokens → `ϱ(r)`.

This talk: which tokenization scheme? Before training anything, characterize each candidate's reconstruction-error floor.

What we're aiming at

- **ChargE3Net** (Koker et al, npj Comp Mat 2024): **0.52% NMAE** on Materials Project — published SotA.
- **electrAI** (OA in-house 3D ResUNet): **2.60% NMAE** validation (Jan 2026 monthly review, 100-epoch run) — stepping-stone target.

Both numbers are achieved losses, not tokenizer floors.

Floor vs. achieved loss

```
total_NMAE = reconstruction_floor + prediction_error
```

The floor is what `encode → decode` alone loses, before any model runs. For a tokenizer to be *competitive* with a target, its floor must sit well below that target — not near it.

- Floor ≈ 2.6% → zero budget for the model vs electrAI.
- Floor ≈ 0.5% → zero budget vs ChargE3Net.
- Floor << target → there's room to train something useful.

Seven candidate tokenization schemes

In this sweep (easy three):

- **1. Direct** — float-per-voxel baseline (≈ lossless).
- **3. Cutoff** — keep top-density voxels, zero the rest.
- **5. Fourier** — keep low-|G| coefficients, drop the rest.

Next:

- **4. Δρ** — subtract an analytic promolecule density, tokenize the residual. Composes with 1/3/5.

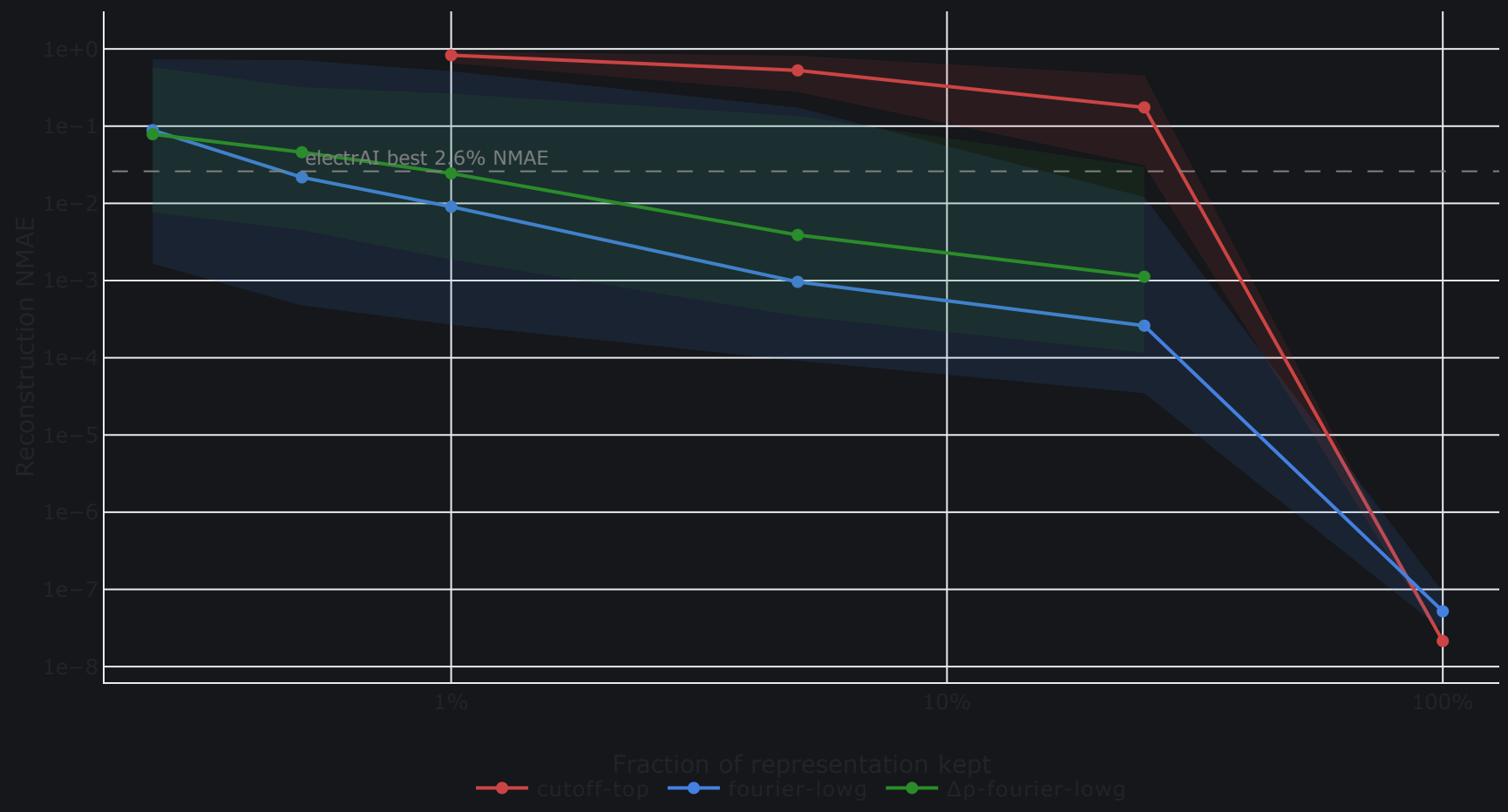
Deferred: 2 (VQ-VAE), 6 (spherical harmonics), 7 (Gaussian/RI fit).

Empirical sweep

- **Data:** 50 Materials Project CHGCARs (electrAI's curated 2,885 subset, first 50 alphabetical).
- **Grid:** 128^3 voxels per sample.
- **Configs:** 16 — cutoff × {1,5,25,100}%, Fourier × {0.25,0.5,1,5,25,100}%, Δρ-Fourier × {0.25,0.5,1,5,25}%.
- **Metrics:** NMAE, χ^2, Hellinger, JSD, Weighted MAE — all on `encode → decode` roundtrip.

Live data: sweep-n50.csv · fidelity_sweep.py.

Reconstruction floor vs. fraction of representation kept



Cutoff (scheme 3) is disqualified

At 25% of voxels kept, median NMAE is still **18%** — already **7×** over electrAI's achieved loss before any model runs.

Why: top-density voxels sit near nuclei. Those are trivially reconstructible from atomic positions. The cutoff scheme keeps the easy part and throws away the hard part (bonding regions).

Fourier (scheme 5) clears the bar

- At 5% coefs: median NMAE **0.1%** — ~3 orders of magnitude below cutoff.
- Leaves ~2.5% budget for the model vs electrAI.
- Mean NMAE is noisier (0.9%) — driven by the oxide tail.

Oxides are the threshold case

Oxide mean NMAE at 5% coefs: **2.4%** — 10–50× worse than every other material category.

Why: O 2p orbitals concentrate density at sharp cusps. Sharp cusps → high-|G| spectral content → a low-pass filter truncates exactly the content that matters.

This is the concrete argument for pursuing scheme 4 (Δρ).

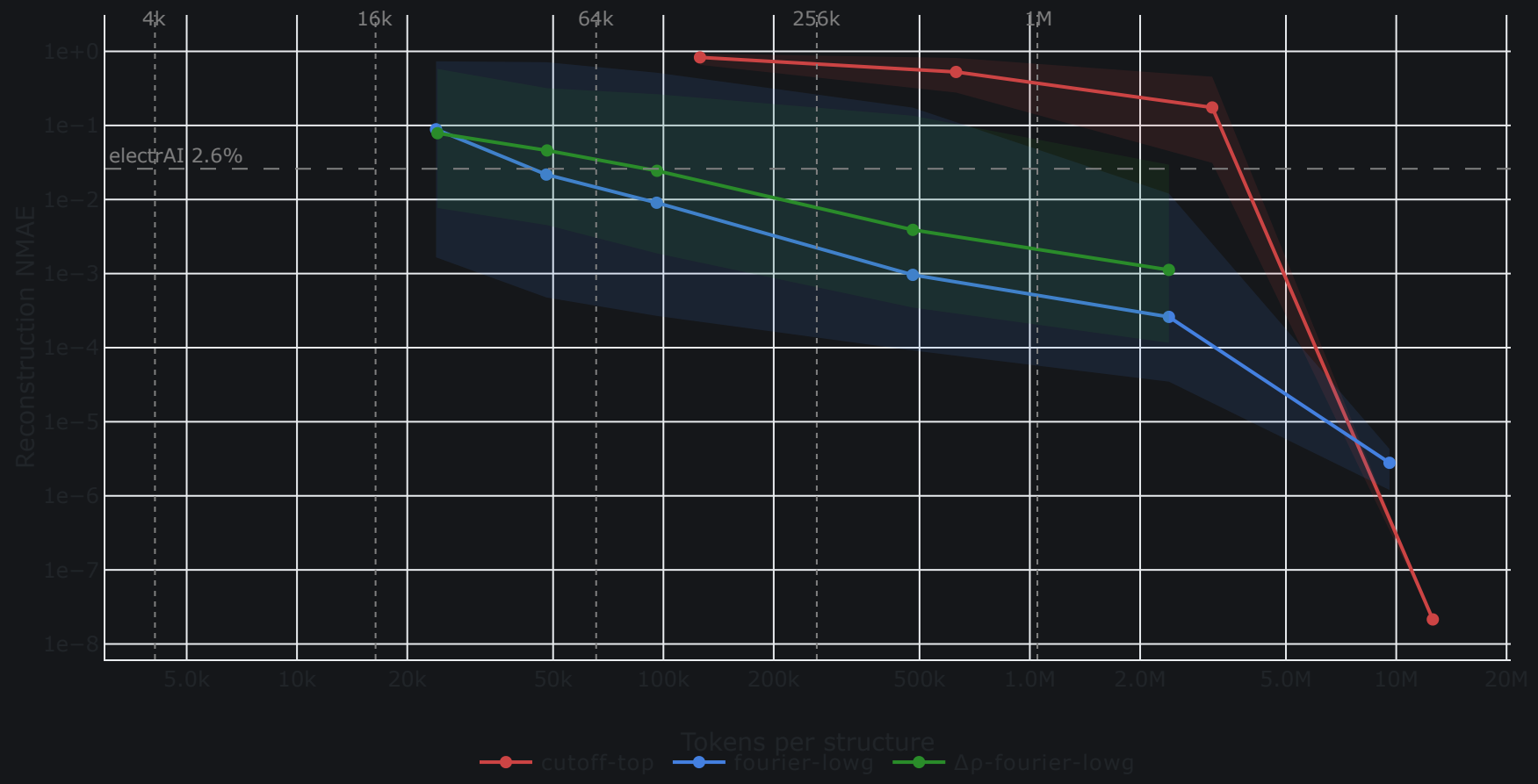
NMAE isn't everything

Yael (Jan 2026 review): NMAE is dominated by high-density voxels near nuclei — a scheme can score well on NMAE while getting the chemically interesting bonding regions wrong.

Sweep reports 5 metrics in parallel: NMAE, χ^2, Hellinger, JSD, Weighted MAE. χ^2 is more sensitive to low-density regions and reshuffles the ranking somewhat.

Try the metric toggle on the live dashboard.

NMAE floor vs. tokens per structure



Every scheme here assumes FP16 codec fidelity (3 tokens per real value, 6 per complex). Vertical lines = 4k / 16k / 64k / 256k / 1M context.

Pivot: patches, not full grids

At native resolution (40^3 to 448^3 rho_gga grids) no single-token-per-voxel scheme fits 8–16k context. Downsampling to 32^3 / 64^3 throws away exactly the information the DFT solver spent hours computing.

New direction: one training example = one `P × P × P` sub-cube of a material's native-resolution grid, prefixed with the material's atomic inventory + the patch's offset within the parent grid.

- **Any voxel is a valid anchor.** Crystals are periodic, so the tokenizer extracts patches with PBC wrap — no "edge" to avoid.
- **M patches per material** (default 32) → 4,305 val structures × 32 = ~138k training rows, each a distinct local context.
- **P = 14 → 14^3 × 2 tokens/voxel = 5,488 density tokens;**~200-token preamble; total ~5.7k, fits 8k with 2k headroom.
- **Offset seed = free data augmentation;** re-running preprocessing with a different RNG seed samples a different set of patches from the same crystals.

What each training example looks like

```
[BOS]
[GRID_START]   nx ny nz          [GRID_END]    # parent grid shape
[ATOMS_START]  Z₁ Z₂ … Zₙ        [ATOMS_END]   # atomic numbers
[POS_START]    ⟨x₁ y₁ z₁⟩ …       [POS_END]     # frac coords (3-byte codec)
[SHAPE_START]  P P P              [SHAPE_END]   # patch dims
[OFFSET_START] ix iy iz           [OFFSET_END]  # patch anchor (low corner)
[HI_START]     hx hy hz           [HI_END]      # wrapped high corner
[DENS_START]   d₀ d₁ … d_{P³−1}   [DENS_END]    # density (2-token 9+12 codec)
[EOS]
```

Vocab (6,792 total): 18 specials + 118 atomic Zs + 1,024 ints + 1,024 position-codec + 4,608 density-codec.

Why `[HI_START]`? Encodes `(ix + P − 1) mod nx` per axis. On any axis where `hi < lo` the patch crossed the PBC boundary. Derivable from `(grid, offset, P)` via modular arithmetic, but making it an explicit observable at layer 1 saves the model from learning it — 5 tokens out of ~5,700.

Example: NaCl patch, P=4, 16^3 grid, offset=(14, 2, 5)

```
  [0]      1  [BOS]
  [1]      7  [GRID_START]
  [2]    152  int = 16 (nx)
  [3]    152  int = 16 (ny)
  [4]    152  int = 16 (nz)
  [5]      8  [GRID_END]
  [6]      3  [ATOMS_START]
  [7]     28  atom Z = 11 (Na)
  [8]     34  atom Z = 17 (Cl)
  [9]      4  [ATOMS_END]
 [10–      …  POS block — 2 atoms × 3 coords × 3 tokens/coord
  29]
 [30]      9  [SHAPE_START]
 [31–    140  int = 4, 4, 4 (P=4 cube)
  33]
 [34]     10  [SHAPE_END]
 [35]     11  [OFFSET_START]
 [36]    150  int = 14 (ix)
 [37]    138  int = 2 (iy)
 [38]    141  int = 5 (iz)
 [39]     12  [OFFSET_END]
 [40]     13  [HI_START]
 [41]    137  int = 1 (hx — WRAPS: 1 < 14)
 [42]    141  int = 5 (hy — no wrap: 5 > 2)
 [43]    144  int = 8 (hz — no wrap: 8 > 5)
 [44]     14  [HI_END]
 [45–      …  DENS block — 4³ = 64 voxels × 2 tokens
 174]
[175]      2  [EOS]
```

Total length 176 tokens. At production (P=14) the DENS block dominates: 5,488 of ~5,700 tokens. This patch anchor `(14, 2, 5)` *wraps on the x axis only — visible at* `[41]` *as* `hx=1 < lo_x=14`*.*

Where we are now

- **Pivoted to patch tokenization** — each training example is a P^3 sub-cube of a material's native-resolution density grid, prefixed with atomic inventory + grid/patch/offset metadata. Resolves the 16k-context constraint without downsampling.
- **30M Qwen3 scaffold** — 6 layers, hidden=512, 8k context, 6,792-token vocab. Target hello-world: 100 steps, monotonically decreasing loss.
- **Wrap-aware tokens** — `[HI_START]` block encodes the wrapped high corner per axis; `hi < lo` directly flags PBC wrap to the model without requiring it to learn modular arithmetic.
- **Data pipeline** — `tomat-rho-gga` Modal volume seeded with 4,305 native-resolution val-split Zarrs (~22 GB); patch tokenizer + parquet preprocessor shipped.

Next

1. **Smoke training on Modal** — tokenize ~128 val structures → 30M Qwen3 on one A100 for 100 steps. Immediate-access compute surface to close the loop end-to-end.
2. **Sweep over (codec × patch size × M × N)** — preprocess + training grids defined; 6 of 9 (codec, P) combos fit 8k context.
3. **Scale to Marin + GCP TPU Research Cluster** — real training run on v5p-8 once the smoke loop is green. Scaffold already in place.
4. **Full train split** — smoke uses val-only (4,305 structures); train split adds 77k structures / ~390 GB.

Where to look

- Live dashboard: tomat.oa.dev
- Repo: Open-Athena/tomat
- Sweep CSV: results/sweep-n50.csv
- Specs: 04-patch-training, 05-modal-smoke, 02-fidelity-sweep (done)
- Notes: docs/discussion-notes.md